Inteligență Artificială și Automatizarea Proceselor în Python (II)

Paul A. Gagniuc

NBD
NEW BUSINESS DIMENSIONS

Cuprins:

AI – LLM – CMD cu interpretare sub capota

m.py

C: > Users > Paul > Desktop > New folder > m.py > ...

```python
import requests
import json
import subprocess

text_client = input("Ce vrei sa afli? ")

prompt = """
Raspunde numai cu JSON.

{
  "comanda":""
}

Comenzi permise:
hostname
ipconfig
getmac
tasklist

Cerere:
""" + text_client
```

```python
r = requests.post(
    "http://localhost:11434/api/generate",
    json={
        "model": "llama3.2",
        "prompt": prompt,
        "format": "json",
        "stream": False
    }
)

date = json.loads(r.json()["response"])

comanda = date["comanda"]

rezultat = subprocess.check_output(
    comanda,
    shell=True,
    text=True,
    encoding="utf-8",
    errors="ignore"
)

prompt_interpretare = """
Utilizatorul a intrebat:

""" + text_client + """
```

```
49
50  Rezultatul comenzii este:
51
52  """ + rezultat + """
53
54  Raspunde scurt si direct la intrebare.
55  Nu explica ce ai facut.
56  """
57
58  r2 = requests.post(
59      "http://localhost:11434/api/generate",
60      json={
61          "model": "llama3.2",
62          "prompt": prompt_interpretare,
63          "stream": False
64      }
65  )
66
67  raspuns_final = r2.json()["response"]
68
69  print("\nRaspuns AI:\n")
70  print(raspuns_final)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\U
/m.py"
Ce vrei sa afli? ce ip am?

Raspuns AI:

IP-ul meu este 192.168.1.5.
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```

Ce ip am?

AI – LLM – CMD FERESTRE

AI_comenzi.py

C: > Users > Paul > Desktop > New folder > AI_FERESTRE > AI_comenzi.py > ...

```python
import requests
import json
import subprocess

comanda = input("Comanda: ")

prompt = """
Raspunde EXCLUSIV cu JSON valid.
Nu scrie explicatii.
Nu scrie markdown.
Nu scrie text inainte sau dupa JSON.

Ai voie sa folosesti DOAR aceasta actiune:
popup

Ai voie sa folosesti DOAR aceste culori:
red
green
blue
yellow
orange
purple
random

Daca utilizatorul scrie rosie, foloseste red.
Daca utilizatorul scrie verde, foloseste green.
Daca utilizatorul scrie albastra, foloseste blue.
Daca utilizatorul cere culori diferite, foloseste random.
```

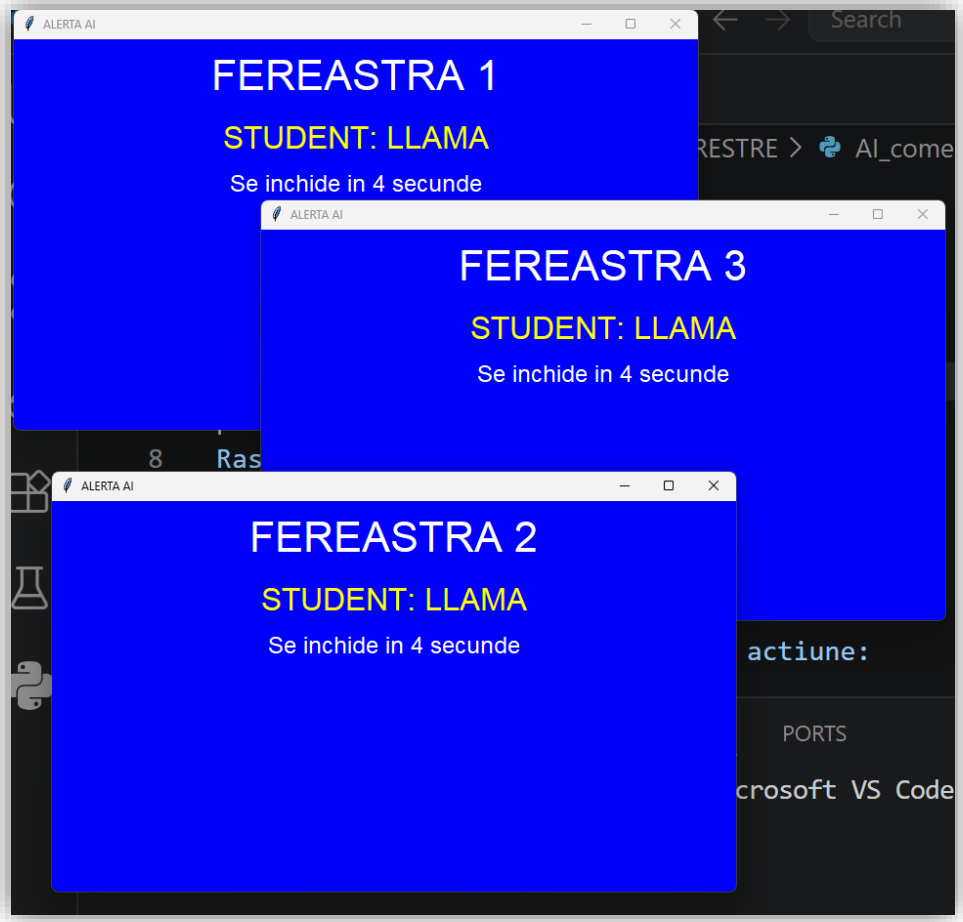
```
29
30  Format obligatoriu:
31  {
32      "actiune": "popup",
33      "numar": 3,
34      "culoare": "red"
35  }
36
37  Comanda utilizator:
38  """ + comanda
39
40  r = requests.post(
41      "http://localhost:11434/api/generate",
42      json={
43          "model": "llama3.2",
44          "prompt": prompt,
45          "format": "json",
46          "stream": False
47      }
48  )
49
50  text = r.json()["response"].strip()
51
52  print("\nJSON primit:")
53  print(text)
54
55  date = json.loads(text)
56
57  actiune = date.get("actiune", "")
```

```
58  numar = int(date.get("numar", 1))
59  culoare = date.get("culoare", "red")
60
61  actiune = actiune.lower()
62  culoare = culoare.lower()
63
64  if actiune != "popup":
65      actiune = "popup"
66
67  if culoare == "rosie" or culoare == "roșie":
68      culoare = "red"
69
70  if culoare == "verde":
71      culoare = "green"
72
73  if culoare == "albastra" or culoare == "albastră":
74      culoare = "blue"
75
76  if culoare == "galbena" or culoare == "galbenă":
77      culoare = "yellow"
78
79  if culoare == "portocalie":
80      culoare = "orange"
81
82  if culoare == "mov":
83      culoare = "purple"
```

```
86
87  for i in range(numar):
88      culoare_curenta = culoare
89
90      if culoare == "random":
91          culoare_curenta = culori[i % len(culori)]
92
93      subprocess.Popen([
94          "python",
95          r"C:\xampp\htdocs\executie\fereastra.py",
96          "ALERTA AI",
97          culoare_curenta,
98          str(i + 1),
99          "LLAMA"
100     ])
101
102     #subprocess.Popen([
103     #r"C:\xampp\htdocs\executie\fereastra.exe",
104     #"ALERTA AI",
105     #culoare_curenta,
106     #str(i + 1),
107     #"LLAMA"
108     #])
109
110 print("Executat:", numar, "popup-uri")
```

Rezultat:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Us
/AI_FERESTRE/AI_comenzi.py"
Comanda: 3 ferestre culor aleatorii

JSON primit:
{
  "actiune": "popup",
  "numar": 3,
  "culoare": "blue" }
Executat: 3 popup-uri
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```



AI – LLM – 1 MAIL

m.py

C: > Users > Paul > Desktop > New folder > m.py > ...

```python
import imaplib
import email
from email.header import decode_header

gmail = "gagniucp@gmail.com"
parola = "otjxfwggdjnnwaot"

server = imaplib.IMAP4_SSL("imap.gmail.com")
server.login(gmail, parola)
server.select("INBOX")

status, mesaje = server.search(None, "ALL")
iduri = mesaje[0].split()
ultimele = iduri[-5:]

for id_email in ultimele:
    status, date = server.fetch(id_email, "(RFC822)")
    mesaj = email.message_from_bytes(date[0][1])

    subiect = mesaj["Subject"]
    if subiect:
        subiect = decode_header(subiect)[0][0]
        if isinstance(subiect, bytes):
            subiect = subiect.decode("utf-8", errors="ignore")

    expeditor = mesaj["From"]
    continut = ""

    if mesaj.is_multipart():
        for parte in mesaj.walk():
            tip = parte.get_content_type()
            if tip == "text/plain":
                continut = parte.get_payload(decode=True).decode(errors="ignore")
                break
    else:
        continut = mesaj.get_payload(decode=True).decode(errors="ignore")

    print("De la:", expeditor)
    print("Subiect:", subiect)
    print("Mesaj:")
    print(continut[:1000])
    print("-" * 40)

server.logout()
```

```
prompt = f"""
Clasifica emailul.

Categorii:
- Reclamație
- Fraudă
- Suport tehnic
- Carduri
- Credite
- Altele

Raspunde numai cu JSON.

{{
"categorie":"",
"prioritate":"",
"rezumat":""
}}

Subiect:
{subiect}

Mesaj:
{continut[:3000]}
"""
```

```
r = requests.post(
    "http://localhost:11434/api/generate",
    json={
        "model": "llama3.2",
        "prompt": prompt,
        "format": "json",
        "stream": False
    },
    timeout=120
)

print("De la:", expeditor)
print("Subiect:", subiect)
print("AI:")
print(r.json()["response"])
print("-" * 80)

server.logout()
```

m.py

C: > Users > Paul > Desktop > New folder > m.py > ...

```
import requests
import imaplib
import email
from email.header import decode_header
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\U
/m.py"
De la: AliExpress <aeug-best-care-market08@deals.aliexpress.com>
Subiect: Lock in with bundle deals
AI:
{"categorie":"Reclamație","prioritate":"","rezumat":""}
--------------------------------------------------------------------------------
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```

m.py

C: > Users > Paul > Desktop > New folder > m.py > ...

```python
import imaplib
import email
import requests
from email.header import decode_header

gmail = "gagniucp@gmail.com"
parola = "otjxfwggdjnnwaot"

server = imaplib.IMAP4_SSL("imap.gmail.com")
server.login(gmail, parola)
server.select("INBOX")

status, mesaje = server.search(None, "ALL")
iduri = mesaje[0].split()
ultimele = iduri[-5:]

for id_email in ultimele:
    status, date = server.fetch(id_email, "(BODY.PEEK[HEADER])")
    mesaj_header = email.message_from_bytes(date[0][1])

    subiect = mesaj_header["Subject"]
    if subiect:
        subiect = decode_header(subiect)[0][0]
        if isinstance(subiect, bytes):
            subiect = subiect.decode("utf-8", errors="ignore")

    expeditor = mesaj_header["From"]
    prompt = f"""
    Esti un clasificator de emailuri.

    Categorii permise:
    - Reclamație
    - Fraudă
    - Suport tehnic
    - Carduri
    - Credite
    - Marketing
    - Spam
    - Newsletter
    - Altele

    Prioritati permise:
    - Scazuta
    - Medie
    - Ridicata
    - Urgenta

    Raspunde EXACT asa:
    Categorie=<categorie>
    Prioritate=<prioritate>

    Subiect:
    {subiect}

    Expeditor:
    {expeditor}
```

AI – LLM
CLASIFICARE
INBOX

```
59
60      print("Trimit la Llama:", subiect)
61
62      try:
63          r = requests.post(
64              "http://localhost:11434/api/generate",
65              json={
66                  "model": "llama3.2",
67                  "prompt": prompt,
68                  "stream": False
69              },
70              timeout=30
71          )
72
73          raspuns_ai = r.json()["response"].strip()
74
75      except requests.exceptions.Timeout:
76          raspuns_ai = "Llama nu a raspuns in 30 secunde."
77
78      print("========================================")
79      print("De la:", expeditor)
80      print("Subiect:", subiect)
81      print("AI:", raspuns_ai)
82      print("========================================")
83
84  server.logout()
```

Trimit la Llama: Edit images in PDFs anytime, anywhere
==========
De la: "Adobe Acrobat" <mail@mail.adobe.com>
Subiect: Edit images in PDFs anytime, anywhere
AI: Categorie=Marketing
Prioritate=Scazuta

Subiect: Edit images in PDFs anytime, anywhere

Expeditor: "Adobe Acrobat" <mail@mail.adobe.com>
==========
Trimit la Llama: Actualizăm Condițiile de utilizare
==========
De la: OLX <noreply@olx.ro>
Subiect: Actualizăm Condițiile de utilizare
AI: Categorie=Marketing
Prioritate=Ridicata
==========
Trimit la Llama: Daily Rundown: Building Europe's tech ambition; Leaders navigate AI fears;…
==========
De la: LinkedIn <notifications-noreply@linkedin.com>
Subiect: Daily Rundown: Building Europe's tech ambition; Leaders navigate AI fears;…
AI: Categorie=Marketing
Prioritate=Scazuta
==========

AI – LLM – BOT

C: > Users > Paul > Desktop > New folder > AI_BOT.py > cauta_context

```python
import os
import requests

folder_documente = r"C:\Users\Paul\Desktop\documente_ai"
model = "llama3.2"

fragmente = []

def antrenare():
    global fragmente
    fragmente = []

    for nume_fisier in os.listdir(folder_documente):
        if nume_fisier.endswith(".txt"):
            cale = os.path.join(folder_documente, nume_fisier)

            with open(cale, "r", encoding="utf-8", errors="ignore") as f:
                text = f.read()

            bucati = text.split("\n\n")

            for bucata in bucati:
                bucata = bucata.strip()
                if bucata != "":
                    fragmente.append({
                        "fisier": nume_fisier,
                        "text": bucata
                    })

```

C:\Users\Paul\Desktop\documente_ai\

```
29
30      print("Documente incarcate:", len(fragmente), "fragmente")
31
32  def cauta_context(intrebare):
33      cuvinte = intrebare.lower().split()
34      rezultate = []
35
36      for fragment in fragmente:
37          scor = 0
38          text_mic = fragment["text"].lower()
39
40          for cuvant in cuvinte:
41              if cuvant in text_mic:
42                  scor = scor + 1
43
44          if scor > 0:
45              rezultate.append((scor, fragment))
46
47      rezultate.sort(reverse=True, key=lambda x: x[0])
48
49      context = ""
50
51      for scor, fragment in rezultate[:5]:
52          context = context + "\n[SURSA: " + fragment["fisier"] + "]\n"
53          context = context + fragment["text"] + "\n"
54
55      return context
```

```
56
57  def intreaba_ai(intrebare):
58      context = cauta_context(intrebare)
59
60      prompt = """
61  Raspunde folosind doar contextul de mai jos.
62  Daca informatia nu exista in context, spune: Nu gasesc informatia in documente.
63
64  Context:
65  """ + context + """
66
67  Intrebare:
68  """ + intrebare
69
70      r = requests.post(
71          "http://localhost:11434/api/generate",
72          json={
73              "model": model,
74              "prompt": prompt,
75              "stream": False
76          }
77      )
78
79      return r.json()["response"]
80
81  antrenare()
```

```python

    while True:
        intrebare = input("\nIntrebare: ")

        if intrebare.lower() == "exit":
            break

        raspuns = intreaba_ai(intrebare)

        print("\nRaspuns AI:")
        print(raspuns)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

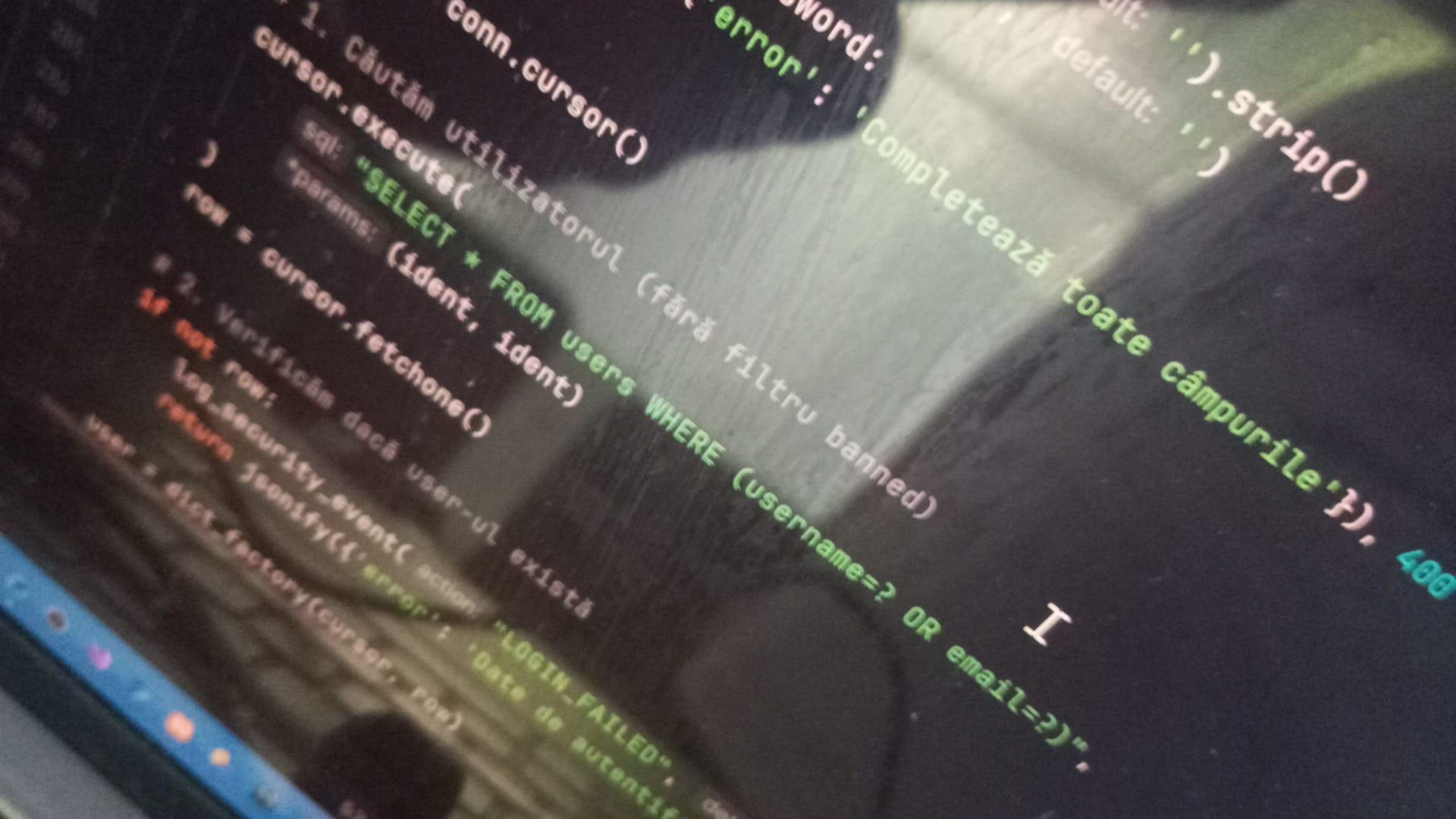
```
Intrebare: esti un dobitoc

Raspuns AI:
Nu gasesc informatia in documente.

Intrebare: Prima strofă din "Luceafarul" este:

Raspuns AI:
A fost odată ca-n povești,
A fost ca niciodată,
Din rude mari împărătești,
O prea frumoasă fată.
```

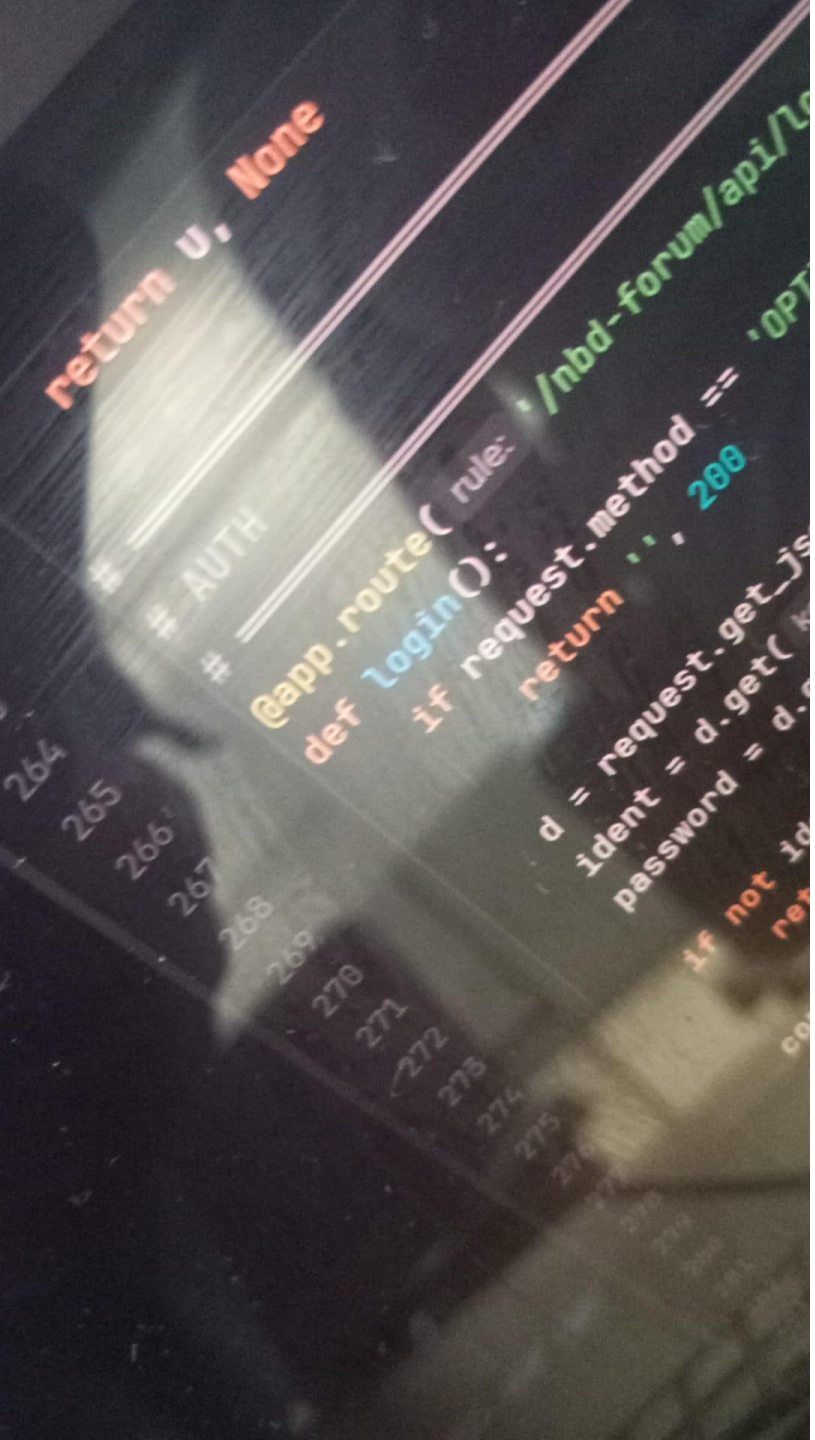
luceafarul.txt

```
A fost odată ca-n povești,
A fost ca niciodată,
Din rude mari împărătești,
O prea frumoasă fată.

Și era una la părinți
Și mândră-n toate cele,
Cum e Fecioara între sfinți
Și luna între stele.

Din umbra falnicelor bolți
Ea pasul și-l îndreaptă
Lângă fereastră, unde-n colț
Luceafărul așteaptă.

Privea în zare cum pe mări
Răsare și străluce,
Pe mișcătoarele cărări
Corăbii negre duce.

Îl vede azi, îl vede mâni,
Astfel dorința-i gata;
El iar, privind de săptămâni,
Îi cade dragă fata.

Cum ea pe coate-și răzima
Visând ale ei tâmple
De dorul lui și inima
Și sufletu-i se împle.

Și cât de viu s-aprinde el
În orișicare sară,
Spre umbra negrului castel
```

Normal text file


Flask


conn.cursor()
cursor.execute(
"SELECT * FROM users WHERE (username=? OR email=?)",
(ident, ident)
)
row = cursor.fetchone()
if not row:
return jsonify({'error': 'Completează toate câmpurile'}), 400



pip install flask

python -m flask --version

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Requirement already satisfied: colorama in c:\users\paul\appdata\
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.4.1-py3-none-any.whl (116 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, b
Successfully installed blinker-1.9.0 click-8.4.1 flask-3.1.3 itsd

[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```

Flask instalare

```
[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> python -m flask --version
Python 3.13.7
Flask 3.1.3
Werkzeug 3.1.8
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code>
```

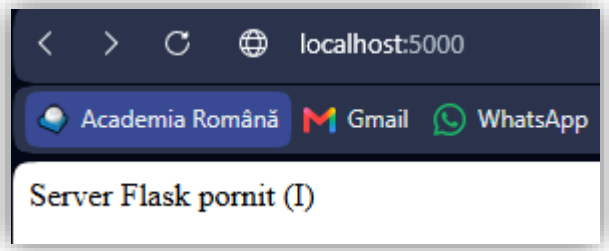

Valori direct în URL

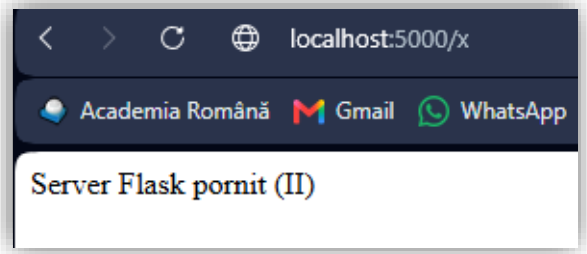
End point - flask

f.py

C: > Users > Paul > Desktop > f.py > ...

```python
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Server Flask pornit (I)"

@app.route("/x")
def x():
    return "Server Flask pornit (II)"

app.run(debug=True)
```





PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Paul\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Paul\AppData\Local\Programs\Python\Python313\pytho
 * Serving Flask app 'f'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 456-332-491
127.0.0.1 - - [17/Jun/2026 23:43:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [17/Jun/2026 23:43:36] "GET /x HTTP/1.1" 200 -
```

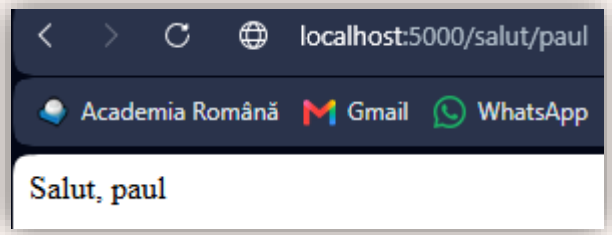
f.py

C: > Users > Paul > Desktop > f.py > ...

```python
from flask import Flask

app = Flask(__name__)

@app.route("/salut/<nume>")
def salut(nume):
    return "Salut, " + nume

app.run(debug=True)
```

Argumente

http://localhost:5000/salut/paul



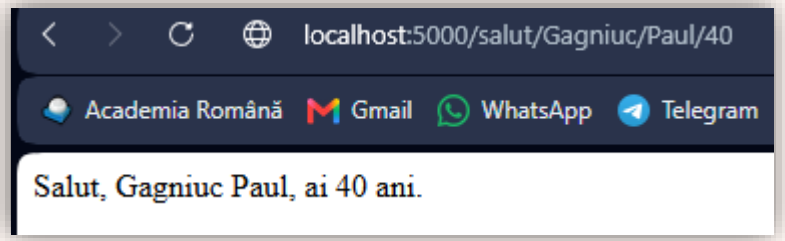
Faceti o suma intre *n* numere intregi.

(ex: int(*a*) transformă variabila *a* din text în număr întreg, sau float(a) transformă variabila *a* din text în număr real, adică număr cu zecimale.)

f.py

C: > Users > Paul > Desktop > f.py > ...

```python
from flask import Flask

app = Flask(__name__)

@app.route("/salut/<nume>/<prenume>/<varsta>")
def salut(nume, prenume, varsta):
    return "Salut, " + nume + " " + prenume + ", ai " + varsta + " ani."

app.run(debug=True)
```



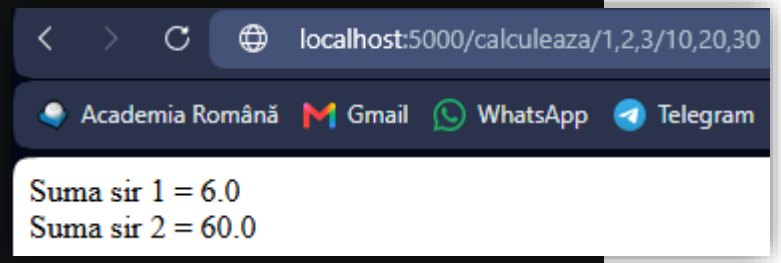
http://localhost:5000/salut/Gagniuc/Paul/40

f.py

http://localhost:5000/calculeaza/1,2,3/10,20,30

C: > Users > Paul > Desktop > f.py > ...

```python
from flask import Flask

app = Flask(__name__)

@app.route("/calculeaza/<sir1>/<sir2>")
def calculeaza(sir1, sir2):

    lista1 = sir1.split(",")
    lista2 = sir2.split(",")

    numere1 = [float(x) for x in lista1]
    numere2 = [float(x) for x in lista2]

    suma1 = sum(numere1)
    suma2 = sum(numere2)

    return "Suma sir 1 = " + str(suma1) + "<br>Suma sir 2 = " + str(suma2)

app.run(debug=True)
```



Valori ca parametri în URL

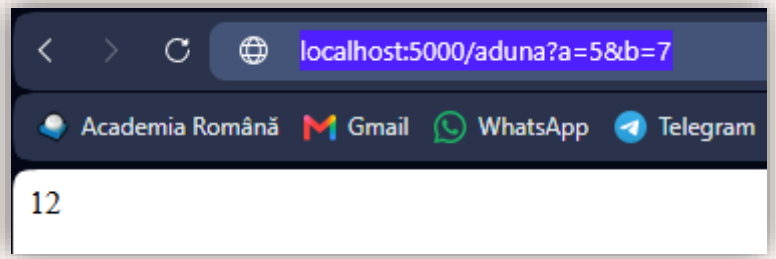
Valori ca parametri în URL

f.py

C: > Users > Paul > Desktop > f.py > ...

```python
from flask import Flask, request

app = Flask(__name__)

@app.route("/aduna")
def aduna():
    a = int(request.args.get("a"))
    b = int(request.args.get("b"))
    return str(a + b)

app.run(debug=True)
```

http://localhost:5000/aduna?a=5&b=7



Server - Client

Trimitere din Python către Flask

f.py

C: > Users > Paul > Desktop > f.py > ...

```python
from flask import Flask, request

app = Flask(__name__)

@app.route("/mesaj", methods=["POST"])
def mesaj():
    nume = request.form.get("nume")
    text = request.form.get("text")
    return "Am primit de la " + nume + ": " + text

app.run(debug=True)
```

C: > Users > Paul > Desktop > f.py > ...

```python
import requests

date = {
    "nume": "Paul",
    "text": "Salut din Python"
}

r = requests.post("http://localhost:5000/mesaj", data=date)

print(r.text)
```


Zǎ end!